Advanced Database Systems

GIN400 – School of Engineering

Chapter 1
Physical Database Storage

1

Outline

- Physical storage
- Disk storage
- Block Address
- Reducing latency for R/W
- Blocking and Spanning
- Indexing
  - Single-level index
  - multi-level index
- Hashing
  - Static hashing
  - Extendable hashing
- Bitmap indexes

Dr. Joseph Zalaket

2

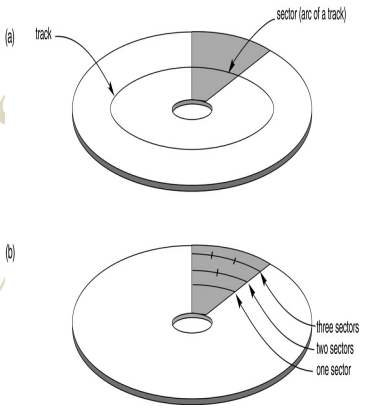
Physical Storage

- The collection of data that makes up a database must be stored physically on some computer storage medium. The DBMS software can then retrieve, update, and process this data as needed. Computer storage media form a *storage hierarchy* that includes two main categories:
  - ***Primary storage:*** This category includes storage media that can be operated on directly by the computer *central processing unit* (CPU), such as the computer main memory and smaller but faster cache memories. Primary storage usually provides fast access to data but is of limited storage capacity *(it is volatile).*
  - ***Secondary storage:*** This category includes magnetic disks, optical disks, and tapes (the 2 latters are sometimes classified as tertiary storage and also called off-line storage). These devices usually have a larger capacity, cost less, and provide slower access to data than do primary storage devices. Data in secondary storage cannot be processed directly by the CPU; it must first be copied into primary storage *(it is non-volatile).*.

3

Disk Storage

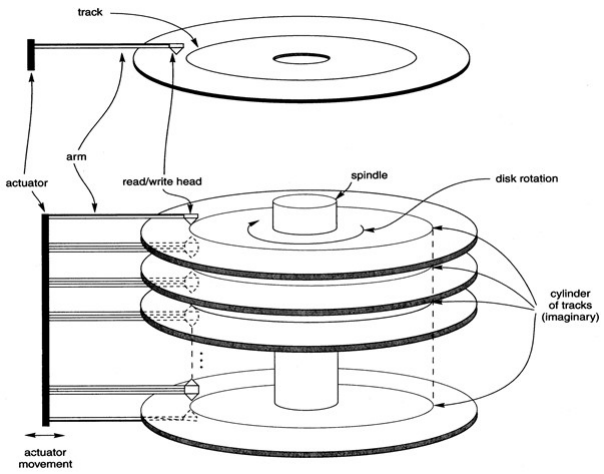
- A disk pack contains several magnetic disks connected to a rotating spindle.
- Disks are divided into concentric circular tracks on each disk surface. Track capacities vary typically from 4 to 50 Kbytes. Each track contains multiple sectors (512 bytes). A sector is the smallest addressing surface .
- A track is divided into equal-sized disk blocks (or pages) (multiple of sectors) by the operating system during disk formatting. Typical block sizes range from *B*=512 bytes (0,5 kbyte) to *B*=4096 bytes (4 kbytes).
- Whole blocks are transferred between disk and main memory for processing.



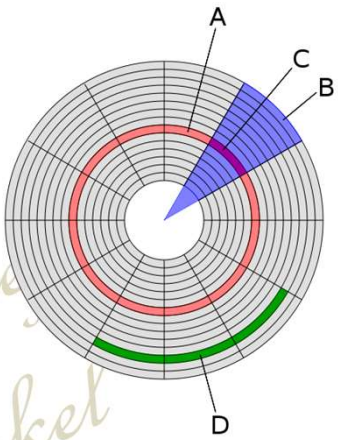
4

Disk read/write

- A *read-write* head moves to the track that contains the block to be transferred. Disk rotation moves the block under the read-write head for reading or writing.



5



Disk structures:
(A) Track
(B) *Geometrical* **sector**
(C) Track sector
(D) Cluster

Dr. Joseph Zalaket

6

Block Address

- A physical disk block address consists of:
  - a ***cylinder number*** (imaginary collection of tracks of same radius from all recorded surfaces),
  - the ***track number*** or surface number (within the cylinder),
  - and ***block number*** (within track).
- In many modern disk drives, a single number called LBA (Logical Block Address) which is a number between 0 and n (assuming the total capacity of the disk is n+1 blocks), is mapped automatically to the right block by the disk drive controller. The address of a buffer (a contiguous reserved area in main storage that holds one block) is also provided.
  - For a read command, the block from disk is copied into the buffer.
  - For a write command, the contents of the buffer are copied into the disk block.
- Sometimes several contiguous blocks, called a cluster, may be transferred as a unit. In this case the buffer size is adjusted to match the number of bytes in the cluster.

I/O Time to Access a Page

- ***Seek latency*** – time to position heads over cylinder containing page (avg = ~10 - 20 ms)
- ***Rotational latency*** – additional time for platters to rotate so that start of block containing page is under head (avg = ~5 - 10 ms)
- ***Transfer time*** – time for platter to rotate over block containing page to transfer it to RAM (depends on size of block)
- ***Latency*** = seek latency + rotational latency

- Our goal – minimize average latency, reduce number of page transfers
- The latency is usually much larger than the block transfer time. To make the transfer of multiple blocks more efficient, it is common to transfer several consecutive blocks on the same track or cylinder.

Reducing latency

- Store pages containing related information close together on disk
  - *Justification*: If application accesses *x*, it will next access data related to *x* with high probability.
  - Page size tradeoff:
    - Large page size – data related to *x* stored in same page; hence additional page transfer can be avoided.
    - Small page size – reduce transfer time, reduce buffer size in main memory.
    - Typical page size – 4096 bytes (4 kbytes).
- Keep cache of recently accessed pages in main memory
  - *Rationale*: request for page can be satisfied from cache instead of disk.
  - Purge pages when cache is full.

Dr. Joseph Zalaket

9

Records

- Data is usually stored in the form of records. Each record consists of a collection of related data values , where each value is formed of one or more bytes.
- Records can be of fixed or variable length
  - Fields of a record may be fixed length or variable length
  - Variable length fields can be mixed into one record: separator characters or length fields are needed so that the record can be parsed.
    (e.g. Name VARCHAR(20), Name='Salem' can be stored as |5|Salem| -- 1 byte for the length and 5 bytes for the value of the field.)

Dr. Joseph Zalaket

10

Blocking

- The records of a file must be allocated to disk blocks because a block is the *unit of data transfer* between disk and memory.
- When the block size is larger than the record size, each block will contain numerous records, although some files may have unusually large records that cannot fit in one block. Suppose that the block size is ***B*** bytes. For a file of fixed-length records of size ***R*** bytes, with ***B*** ≥ ***R***, we can fit ***bfr*** = $\lfloor B/R \rfloor$ records per block ($\lfloor x \rfloor$ floor)
- The value ***bfr*** is called the blocking factor for the file. In general, R may not divide B exactly, so we have some unused space in each block equal to ***B - (bfr * R)*** bytes.

11

Blocking

- Giving a fixed length record of size ***R***=84 bytes, assume ***B*** = 4 kbytes

  $bfr = \lfloor B/R \rfloor = \lfloor 4096/84 \rfloor = 48$ record/bloc
- unused space in each block:

  B - (bfr * R) = 4096 – 48 x 84= 64 bytes
- To locate the record 322 of the file, we have to locate its holding bloc *b*= $\lceil 322/48 \rceil = 7$ ($\lceil x \rceil$ ceiling).
- To transfer the bloc into memory buffer: the first record of bloc 7 has the number 6 x 48 + 1 = 289 and the last 7 x 48 = 336.
- And thus, the record 322 is the 34th on block 7

12

Spanning

- To utilize the unused space, we can store part of a record on one block and the rest on another. A pointer at the end of the first block points to the block containing the remainder of the record in case it is not the next consecutive block on disk. This organization is called ***spanned***, because records can span more than one block.
- Whenever a record is larger than a block, we *must* use a ***spanned*** organization. If records are not allowed to cross block boundaries, the organization is called ***unspanned***.

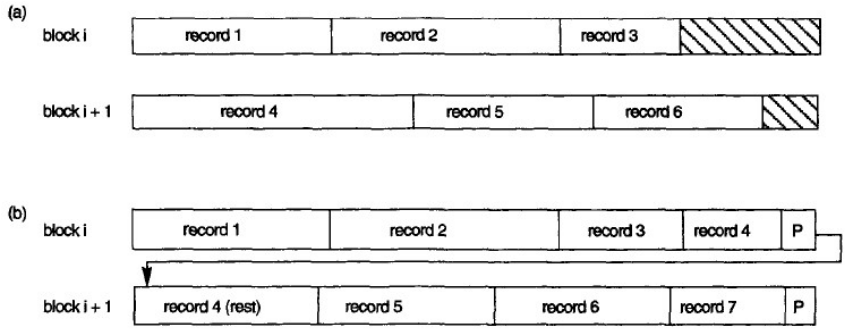
13

Spanning

- For variable-length records using spanned organization, each block may store a different number of records. In this case, the blocking factor *bfr* represents the *average* number of records per block for the file.
- We can use *bfr* to calculate the number of blocks *b* needed for a file of *r* records:
  $b = \lceil r/bfr \rceil$ blocks ($\lceil x \rceil$ ceiling)

14

Unspanned/Spanned



Types of record organization. (a) Unspanned. (b) Spanned.

15

Unspanned/Spanned

- The physical disk blocks that are allocated to hold the records of a file can be *contiguous*, *linked*, or *indexed*.
- In a file of fixed-length records, all records have the same format. Usually, unspanned blocking is used with such files.
- Files of variable-length records require additional information to be stored in each record, such as *separator characters* and *field types*. Usually spanned blocking is used with such files
- Several ways to store variable length records
  - attach an "end of record" symbol (delimiter) to mark the end of each record
  - store the length of the record at the beginning of each record
  - store header information at the beginning of the file with the location and length of each record
  - these techniques can be applied at the file, block, or record level

Dr. Joseph Zalaket

16

Organization of Records in Files

- **Unordered or Heap** – a record can be placed anywhere in the file where there is space (average time to search a record = number of blocks of the file *b/2*)
- **Ordered or Sequential** – store records in sequential order, based on the value of the search key of each record (average time to search a record = *b/2* using sequential scan and $log_2 b$ binary scan)
- **Hashing** – a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed
- Remarks:
  - Records of each relation may be stored in a separate file.
  - In a **multitable clustering file organization** records of several different relations can be stored in the same file
    Motivation: store related records on the same block to minimize I/O

17

Index

- Mechanism for efficiently locating row(s) without having to scan entire table
- A single-level index is an auxiliary file that makes it more efficient to search for a record in the data file.
- The index is usually specified on one field of the file (although it could be specified on several fields)
- One form of an index is a file of entries **<field value, pointer to record>,** which is ordered by field value
- The index is called an *access path* on the field.
- The index file usually occupies considerably less disk blocks than the data file because its entries are much smaller
- Indexes can also be characterized as dense or sparse.
  - A **dense index** has an index entry for *every search key value* (and hence every record) in the data file.
  - A **sparse** (or **nondense**) **index**, on the other hand, has index entries for only some of the search values.

18

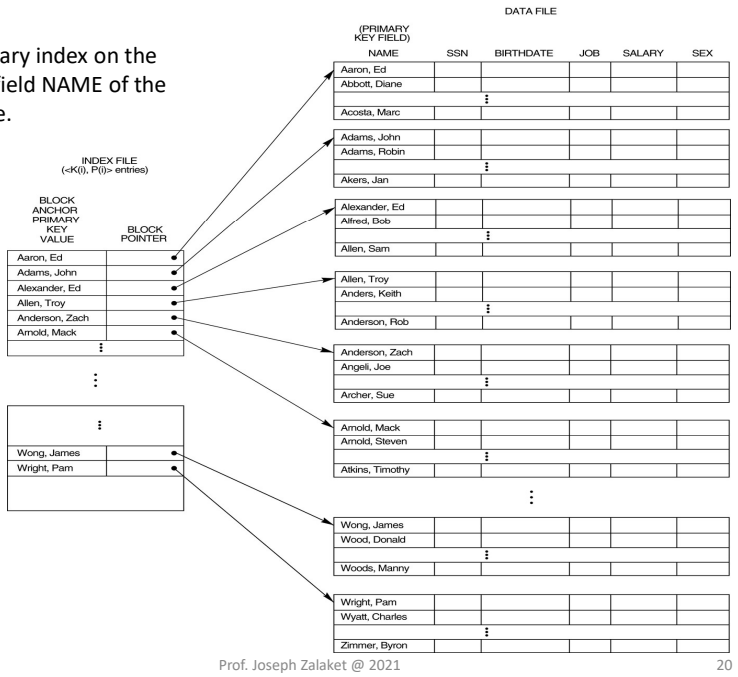
Single-Level Index

- **Primary Index**
  - Defined on an *ordered data file* on a *key field* usually the *primary key* of the data file. It requires that the ordering field of the data file have a distinct value for each record.
  - Includes one index entry *for each block* in the data file; the index entry has the key field value for the *first record* in the block, which is called the *block anchor.*
  - A primary index is a nondense (sparse) index, since it includes an entry for each disk block of the data file and the keys of its anchor record rather than for every search value.

Dr. Joseph Zalaket

19

Example Primary index on the ordering key field NAME of the EMPLOYEE file.



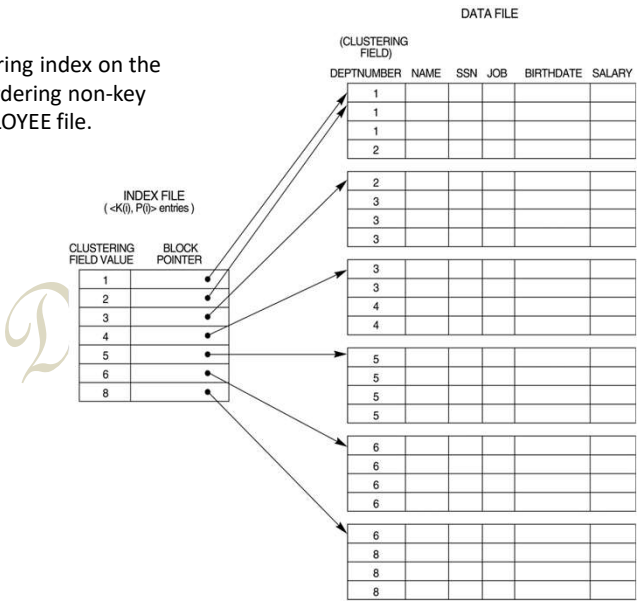
20

Single-Level Index

- **Clustering Index**
  - Defined on an ordered data file on a *non-key field* unlike primary index. It doesn't require distinct value for the ordering field per record.
  - Includes one index entry *for each distinct value* of the field; the index entry points to the first data block that contains records with that field value.
  - It is *nondense* (sparce) index, because it has an entry for every distinct value of the indexing field which is a non-key by definition.
  - *Remark: There can be at most one clustered index on a file as the physical file will be ordered according to it.*

21

Example a clustering index on the DEPTNUMBER ordering non-key field of the EMPLOYEE file.



22

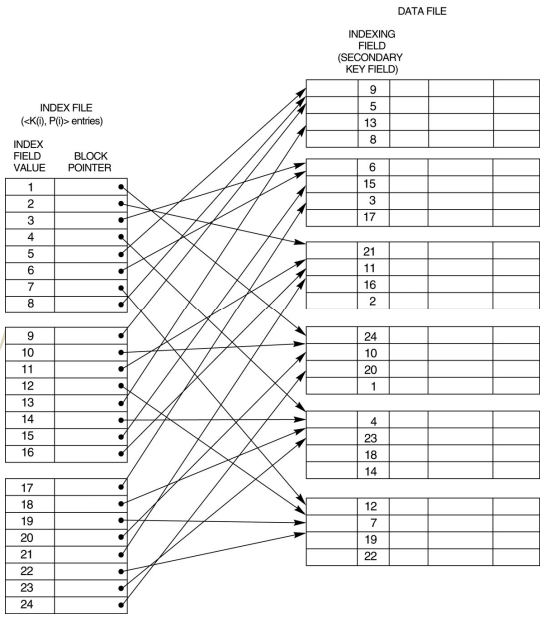
Single-Level Index

- **Secondary Index**
  - A secondary index provides a secondary means of accessing a file for which some primary index already exists.
  - The secondary index may be on a field which is a candidate key and has a unique value in every record in this case it is called a *secondary key*, or a non-key with duplicate values.
  - The index is an ordered file with two fields <K(i), P(i)>.
    - K(i) is of the same data type as some *nonordering field* of the data file that is used as secondary *indexing field.*
    - P(i) is either a *block* pointer or a *record* pointer.
  - A secondary index includes one entry *for each record* in the data file; hence, it is a *dense index*
  - There can be many secondary indexes (and hence, indexing fields) for the same file.

23

A dense <u>secondary unique index</u> (with block pointers) on a nonordering key field of a file.

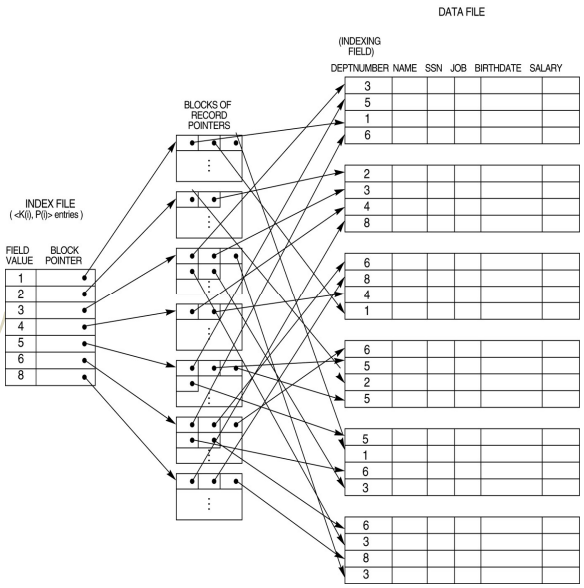
Once the block is transferred to the main memory a search for the desired record within the block is carried out.



24

A secondary index (with record pointers) on a non-key field implemented using one level of indirection so that index entries are of fixed length and have unique field values.

Here, P(i) points to a block of record pointers each of which contains pointers to records having the same value for K(i).



25

Comparison

- A secondary index usually needs more storage space and longer search time than does a primary index, because of its larger number of entries (dense).

Example: Given the following relation (stored as data file):
EMPLOYEE(NAME, SSN, ADDRESS, JOB, SAL, …)
Suppose that:
record size R=150 bytes
block size B=512 bytes
r=30,000 records
Then, we get:
blocking factor $bfr = \lfloor B/R \rfloor = \lfloor 512/150 \rfloor = 3$ records/block

number of file blocks $b = \lceil (r/bfr) \rceil = \lceil (30000/3) \rceil = 10000$ blocks
For an index on the SSN field, assume the field size $V_{SSN}=9$ bytes, assume the block pointer size $P_b=7$ bytes and the block pointer size $P=7$.
Then:
index entry size $R_I=(V_{SSN}+ P_b)=(9+7)=16$ bytes
index blocking factor $bfr_I = \lfloor B/ R_I \rfloor = \lfloor 512/16 \rfloor = 32$ entries/block

26

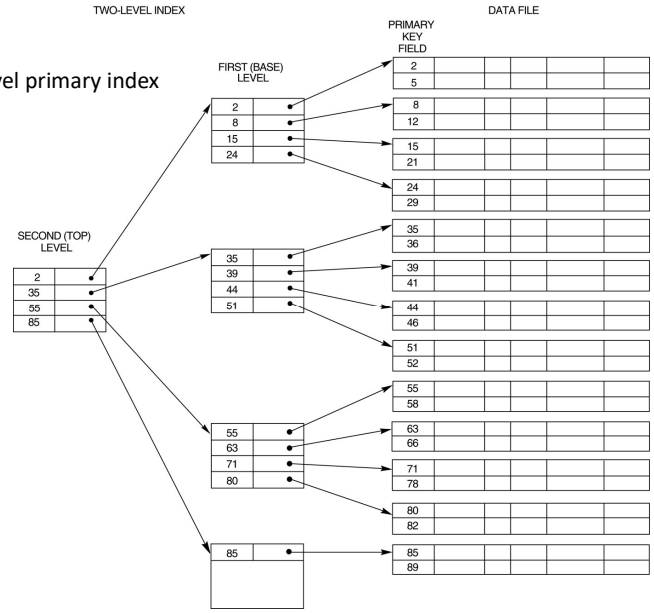
Comparison

- In the case of a primary index (sparse), the total number of index entries r_I is equal to the number of blocks in the data file, which is b=10000.
  number of index blocks $b_I = \lceil (r_I / Bfr_I) \rceil = \lceil (10000/32) \rceil = 313$ blocks
  binary search needs $\log_2 (b_I) = \lceil \log_2 (313) \rceil = \lceil 8,29 \rceil = 9$ block accesses
- In the case of a unique secondary index (dense), the total number of index entries r_I is equal to the number of records in the data file, which is r=30000.
  number of index blocks $b_I = \lceil (r / Bfr_I) \rceil = \lceil (30000/32) \rceil = 938$ blocks
  binary search needs $\log_2 (b_I) = \lceil \log_2 (938) \rceil = \lceil 9.8734 \rceil = 10$ block accesses
- To search for a record using the index, we need an additional block access to the data file for a total of 9 + 1 = 10 (primary) or 11 (secondary) block accesses.
- This is compared to an average linear search cost of:
  (b/2)= 10000/2= 5000 block accesses
- If the file records are ordered, the binary search cost would be:
  $\log_2 b = \log_2 (10000) = 13.287712 = 14$ block accesses

Multi-level Index

- Because a single-level index is an ordered file, we can create a primary index to the index file; in this case, the original index file is called the *first-level index* and the index to the index is called the *second-level index.*
- We can repeat the process, creating a third, fourth, ..., top level until all entries of the *top level* fit in one disk block.
- A multi-level index can be created for any type of first-level index (primary, secondary, clustering) as long as the first-level index consists of *more than one* disk block.
- Remark:
  - Such a multi-level index is a form of *search tree* ; however, insertion and deletion of index entries is a severe problem because every level of the index is an ordered file.

A two-level primary index



29

Multi-level Index

- A binary search requires approximately $log_2(b_I)$ block accesses for an index with b_I blocks, because each step of the algorithm reduces the part of the index file that we continue to search by a factor of 2.
- The idea behind a multilevel index is to reduce the part of the index that we continue to search by bfr_I the blocking factor for the index, which is generally larger than 2. Hence, the search space is reduced much faster.
- The value bfr_I is called the fan-out (***fo***) of the multilevel index
- Searching a multilevel index requires approximately $log_{fo}(b_I)$ block accesses, which is a smaller than for binary search if $fo > 2$.

Dr. Joseph Zalaket

30

Multi-level Index

- The blocking factor *bfr*, for the second level-and for all subsequent levels-is the same (one field value and one block address for all).
- If the first level has r_1 entries, and the blocking factor $bfr_1 = fo$, then the first level needs $b_1 = \lceil r_1 / fo \rceil$ blocks, which is therefore the number of entries r_2 needed at the second level of the index.
- We can repeat this process at the second level, where $r_3 = b_2 = \lceil r_2 / fo \rceil$ blocks, and so on. We can repeat until all the entries of some index level *t* fit in a single block. This block at the tth level is called the top index level. Each level reduces the number of entries at the previous level by a factor of *fo*
- we can use the formula $1 \leq (r_1 / (fo)^t)$ to calculate t. Hence, a multilevel index with r_1 first-level entries will have approximately *t* levels, where $t = log_{fo}(r_1)$.

31

Multi-level Index

- Example: Suppose that the dense secondary index of page-26 is converted into a multilevel index. We calculated the index blocking factor $bfr_I = 32$ (= *fo*) index entries per block
- the number of first-level blocks $b_I = b_1 = 938$ blocks was also calculated.
- The number of second-level blocks will be $b_2 = \lceil b_1 / fo \rceil = \lceil 938/32 \rceil = 30$ blocks, and the number of third-level blocks will be $b_3 = \lceil b_2 / fo \rceil = \lceil 30/32 \rceil = 1$ block. Hence, the third level is the top level of the index, and t =3. To access a record by searching the multilevel index, we must access one block at each level plus one block from the data file, so we need t + 1 = 3 + 1 = 4 block accesses.
- Compare this to the previous example, where 10+1=11 block accesses were needed when a single-level index and binary search were used.

32

B-Tree and B+-Tree

- A multi-level index is a form of *search tree* . Because of the insertion and deletion problem using a search tree (*the tree can become non balanced by having leafs at different levels*), most multi-level indexes use B-tree or B+-tree data structures, which leave space in each tree node (disk block) to allow for new index entries.
- These data structures are variations of search trees that allow efficient insertion and deletion of new search values by keeping the tree balanced (*keeping all the leafs at the same level*).
- In B-Tree and B+-Tree data structures, each tree node corresponds to a disk block.
- Remarks:
  - In a B-tree, pointers to data records exist at all levels of the tree
  - In a B+-tree, all pointers to data records exists at the leaf-level nodes
  - A B+-tree can have less levels (or higher capacity of search values) than the corresponding B-tree

33

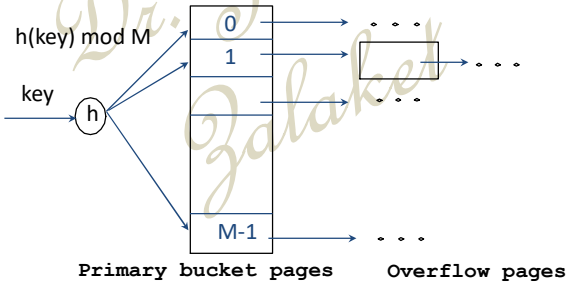
Hash-based indexes

- Hash-based indexes are suitable for equality selections. Cannot support range searches.
- A **bucket** is a unit of storage containing one or more records (a bucket is typically a disk block).
- In a **hash file organization** we obtain the bucket of a record directly from its search-key value using a **hash function.**
- Hash function *h* is a function from the set of all search-key values *K* to the set of all bucket addresses *B.*
- Hash function is used to locate records for access, insertion as well as deletion.
- Records with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate a record.

34

Static Hashing

- Number of primary pages is fixed, allocated sequentially, never de-allocated; we use overflow pages if needed.
- Long overflow chains can degrade performance.
- **h**(*k*) mod M = bucket to which data entry with key *k* belongs. (M = number of buckets)
- **h**(*k*) = (a * *k* + b) usually works well.



35

Static Hashing

- In static hashing, function *h* maps search-key values to a fixed set of *B* of bucket addresses. Databases grow or shrink with time.
  - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
  - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
  - If database shrinks, again space will be wasted.
- Solution: allow the number of buckets to be modified dynamically.

36

Extendable hashing

- Extendable hashing is a form of dynamic hashing
  - Hash function generates values over a large range — typically *b*-bit integers, with *b* = 32.
  - At any time use only a postfix (or prefix) of the hash function to index into a table of bucket addresses.
  - Let the length of the postfix be *i* bits, $0 \leq i \leq 32$.
    - Bucket address table size = 2^{i}. Initially $i = 0$ (2^{32} is over 4 billion).
    - Value of *i* grows and shrinks as the size of the database grows and shrinks.
  - Multiple entries in the bucket address table may point to the same bucket.
  - Thus, actual number of buckets is <= 2^{i}
    - The number of buckets also changes dynamically due to merging and splitting of buckets.

37

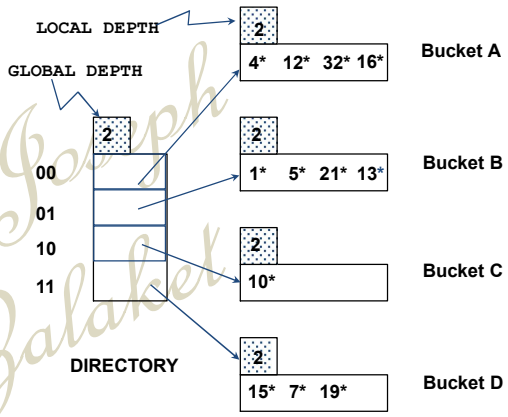
Extendable hashing

- Situation: Bucket (primary page) becomes full. Why not re-organize file by doubling the number of buckets?
  - Problem 1: Reading and writing all pages is expensive!
  - Problem 2: May have many under-utilized pages!
  - Idea: Use directory of pointers to buckets, double the number of buckets by doubling the directory, splitting just the bucket that overflowed!
  - Directory much smaller than file, so doubling it is much cheaper. Only one page of data entries is split. No overflow page!
  - Trick lies in how hash function is adjusted!

38

Extendable hashing

- Directory is array of size $4=2^2$, length of the postfix is 2.
- To find bucket for *r*, take last *`global depth'* number of bits of **h**(*r*);.
- (e.g. If **h**(*r*) = 5 = binary 101, it is in bucket pointed to by 01).



Dr. Joseph Zalaket

39

Extendable hashing

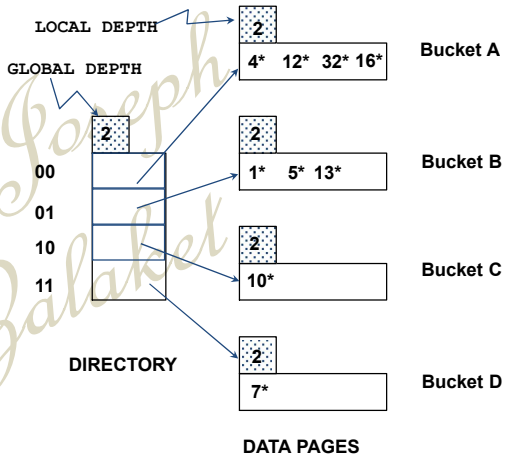
- Insert: If bucket is full, split it (allocate new page, re-distribute).
- If necessary, double the directory. (As we will see, splitting a bucket does not always require doubling; we can tell by comparing global depth with local depth for the split bucket.)

Dr. Joseph Zalaket

40

Extendable hashing

- Directory is array of size $4=2^2$, length of the postfix is 2.

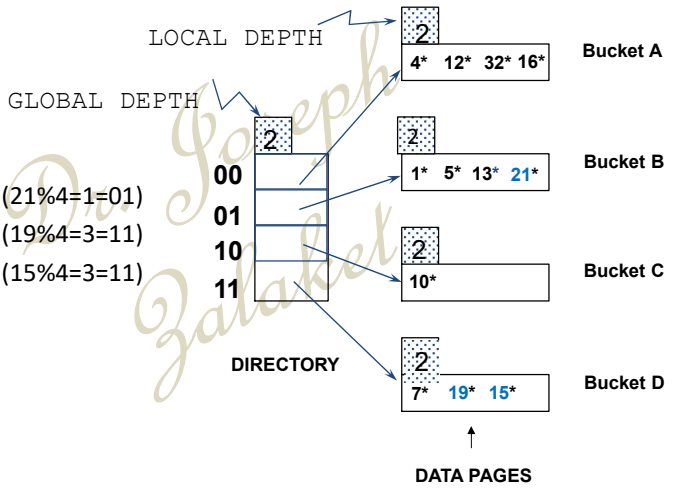


Dr. Joseph Zalaket

41

Extendable hashing

- Insert:
  - 21 = 10101 (21%4=1=01)
  - 19 = 10011 (19%4=3=11)
  - 15 = 01111 (15%4=3=11)

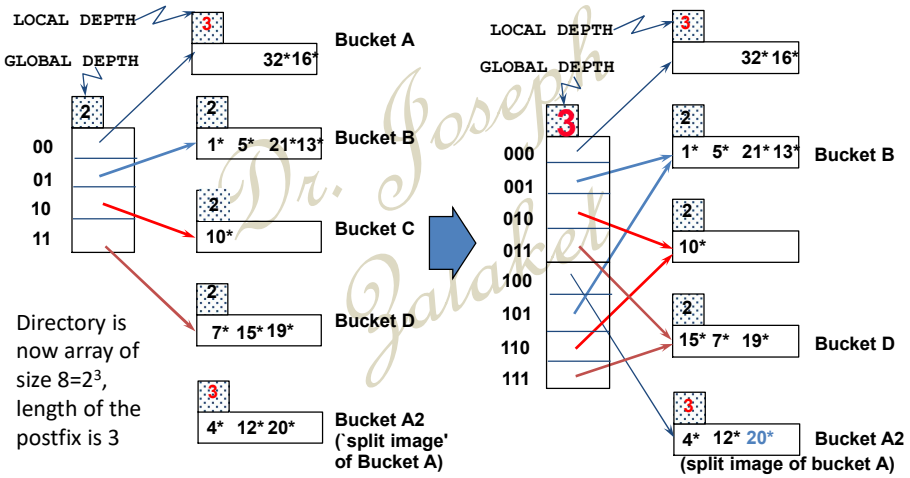


Dr. Joseph Zalaket

42

Extendable hashing

- 20 = 10100 (20%4 = 0, overflow)



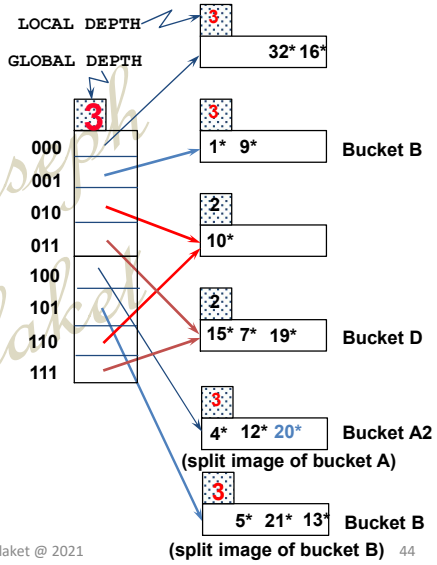
Dr. Joseph Zalaket

43

Extendable hashing

- Insert: 9 = 10001 (9%8=1=001)
- Overflow => splitting

but no doubling as **local depth ≠ global depth.**



Dr. Joseph Zalaket

44

Bitmap Index

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from 0
  - Given a number *n* it must be easy to retrieve record *n*
    - Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values
  - E.g. gender, country, state, …
  - E.g. income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000- infinity)
- A bitmap is simply an array of bits

45

Bitmap Index

- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute
  - Bitmap has as many bits as records
  - In a bitmap for value v, the bit for a record is 1 if the record has the value v for the attribute, and is 0 otherwise

record number	*ID*	*gender*	*income_level*
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income_level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

46

Bitmap Index

- Bitmap indices are useful for queries on multiple attributes
  - not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - Intersection (and)
  - Union (or)
  - Negation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
  - E.g. 100110 AND 110011 = 100010
    100110 OR 110011 = 110111
    NOT 100110 = 011001
  - Males with income level L1: 10010 AND 10100 = 10000
    - Can then retrieve required tuples.
    - Counting number of matching tuples is even faster

Dr. Joseph Zalaket